

Université Hassan 1^{er}

École Nationale des Sciences Appliquées – Berrechid

RAPPORT - Mini projets
ERP - Gestion des Ressources d'Entreprise

Filière : Génie Informatique

Module : Administration des bases de données

Semestre : S7

Réalisé par :
ENNAQUI Zakaria
CHIGUER Abderrahmane

Pr. SAHMI Imane

Année Universitaire : 2025/2026

Table des matières

1	Introduction	3
1.1	Contexte du Projet	3
1.2	Objectifs	3
1.3	Technologies Utilisées	3
2	Conception de la Base de Données	4
2.1	Modèle Conceptuel de Données (MCD)	4
2.1.1	Entités Principales	4
2.1.2	Relations et Cardinalités	4
2.2	Modèle Logique de Données (MLD)	5
2.3	Normalisation	5
3	Implémentation de la Base de Données	5
3.1	Création des Tables	5
3.2	Exemple de Création de Table	5
3.3	Séquences	6
4	Programmation PL/SQL	6
4.1	Procédures Stockées	6
4.1.1	Calcul des Salaires Mensuels	6
4.1.2	Vérification des Stocks	7
4.2	Fonctions	7
4.2.1	Calcul du Total d'une Commande	7
4.2.2	Vérification de la Disponibilité	8
4.3	Package de Gestion des Commandes	8
4.3.1	Spécification du Package	8
4.4	Triggers	9
4.4.1	Trigger d'Audit	9
4.4.2	Trigger de Mise à Jour Automatique	9
4.4.3	Trigger de Validation	10
5	Gestion des Utilisateurs et des Privilèges	11
5.1	Création des Utilisateurs	11
5.2	Création des Rôles	11
5.3	Attribution des Privilèges	11
5.3.1	Privilèges pour le Rôle Administrateur	11
5.3.2	Privilèges pour le Rôle Ventes	12
5.4	Attribution des Rôles aux Utilisateurs	12
5.5	Vues	12
6	Optimisation des Performances	12
6.1	Stratégie d'Optimisation	12
6.2	Indexation	12
6.2.1	Index sur les Clés Étrangères	12
6.2.2	Index Composites	13
6.2.3	Index Basés sur Fonction	13
7	Tests	13
7.1		13
7.1.1	Création de Commande	13
7.1.2	Calcul de Salaire	13
7.1.3	Vérification des Stocks	14

7.2	14
7.2.1 Test des Privilèges	14
8 Conclusion	14
8.1 Objectifs Atteints	14
8.2 Compétences Acquis	14
8.3 Points Forts du Projet	15
8.4 Difficultés Rencontrées et Solutions	15
8.5 Améliorations Possibles	15
8.6 Perspectives	15
8.7 Conclusion	15
Bibliographie	16

1 Introduction

Ce rapport présente le projet d'administration de bases de données réalisé dans le cadre du module *Administration des Bases de Données*. Le projet consiste en la conception, l'implémentation et l'administration d'un système de Gestion des Ressources d'Entreprise utilisant Oracle Database.

1.1 Contexte du Projet

Le système de Gestion des Ressources d'Entreprise développé permet de gérer les aspects suivants d'une entreprise :

- Gestion des employés et des départements
- Gestion des clients et des fournisseurs
- Gestion des produits et des stocks
- Gestion des commandes et des ventes
- Gestion des transactions financières
- Calcul automatisé des salaires avec primes

1.2 Objectifs

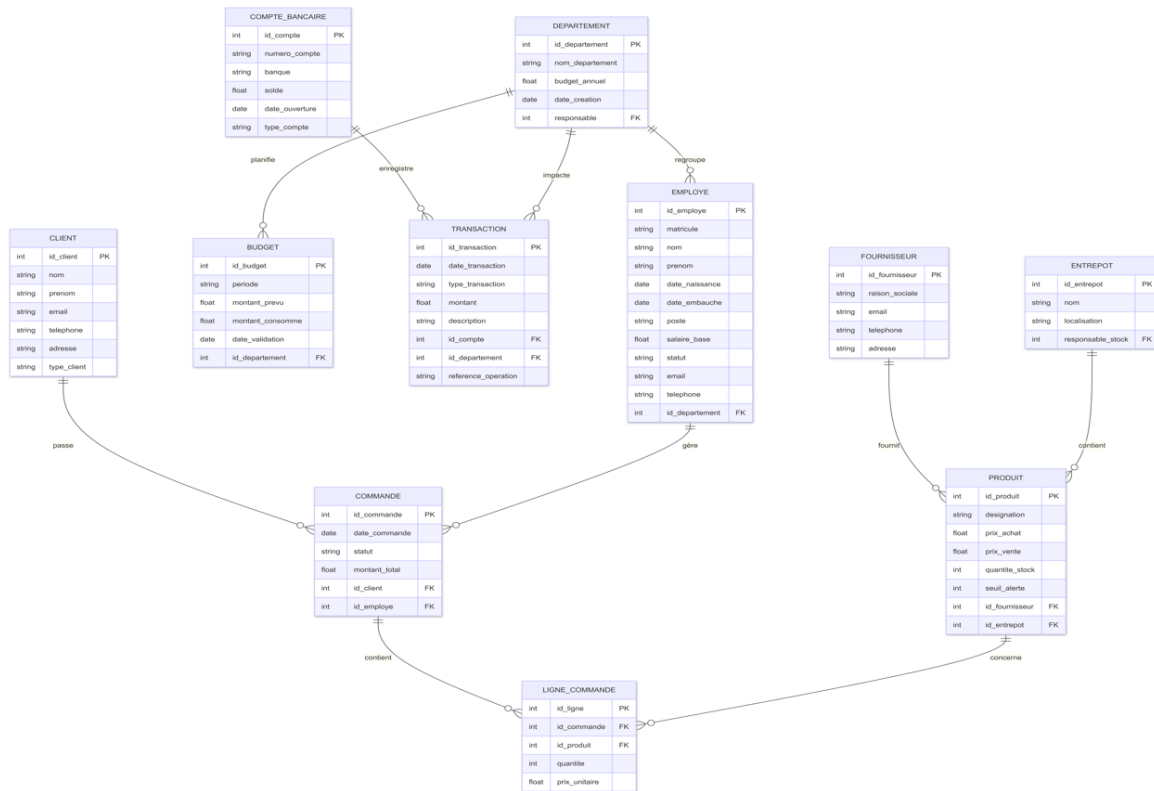
Les objectifs principaux de ce projet sont :

- Concevoir un modèle de données normalisé et cohérent
- Implémenter une base de données Oracle
- Développer des procédures stockées et des fonctions en PL/SQL
- Mettre en place un système de gestion des utilisateurs et des privilèges
- Élaborer une stratégie de sauvegarde et de restauration
- Optimiser les performances de la base de données

1.3 Technologies Utilisées

- **SGBD** : Oracle Database XE
- **Langage** : SQL et PL/SQL
- **Outil de développement** : Oracle SQL Developer
- **Environnement** : Docker

2 Conception de la Base de Données



2.1 Modèle Conceptuel de Données (MCD)

Le modèle conceptuel identifie les entités principales et leurs relations :

2.1.1 Entités Principales

- **DÉPARTEMENT** : Représente les différents départements de l'entreprise
- **EMPLOYÉ** : Contient les informations sur les employés
- **CLIENT** : Gère les informations des clients (particuliers et entreprises)
- **FOURNISSEUR** : Stocke les données des fournisseurs
- **PRODUIT** : Catalogue des produits commercialisés
- **ENTREPÔT** : Lieux de stockage des produits
- **COMMANDE** : Commandes passées par les clients
- **TRANSACTION** : Opérations financières

2.1.2 Relations et Cardinalités

- Un département *emploie* plusieurs employés (1,N)
- Un employé *appartient à* un département (0,1)
- Un client *passer* plusieurs commandes (1,N)
- Une commande *est passée par* un client (1,1)
- Un produit *est fourni par* un fournisseur (0,1)
- Un entrepôt *stocke* plusieurs produits (1,N)
- Une commande *contient* plusieurs produits (N,N)

2.2 Modèle Logique de Données (MLD)

Le passage du MCD au MLD a permis de transformer les entités et associations en tables relationnelles. Les principales transformations effectuées sont :

1. Chaque entité devient une table avec sa clé primaire
2. Les associations (1,N) sont transformées en clés étrangères
3. Les associations (N,N) deviennent des tables de liaison
4. Ajout de contraintes d'intégrité référentielle

2.3 Normalisation

La base de données respecte la troisième forme normale (3NF) :

- **1FN** : Toutes les valeurs sont atomiques
- **2FN** : Élimination des dépendances partielles
- **3FN** : Élimination des dépendances transitives

Les tables PRIMES_PERFORMANCE, PRIMES_ANCIENNETE et PRIMES_DEPARTEMENT ont été séparées pour éviter la redondance et respecter la normalisation.

3 Implémentation de la Base de Données

3.1 Création des Tables

Les tables ont été créées avec les contraintes suivantes :

- Clés primaires pour identifier uniquement chaque enregistrement
- Clés étrangères pour maintenir l'intégrité référentielle
- Contraintes CHECK pour valider les données
- Contraintes NOT NULL pour les champs obligatoires
- Contraintes UNIQUE pour éviter les doublons

3.2 Exemple de Création de Table

```

1 CREATE TABLE EMPLOYE (
2     id_employe NUMBER PRIMARY KEY,
3     matricule VARCHAR2(20) UNIQUE NOT NULL,
4     nom VARCHAR2(50) NOT NULL,
5     prenom VARCHAR2(50) NOT NULL,
6     date_naissance DATE,
7     date_embauche DATE DEFAULT SYSDATE,
8     salaire_base NUMBER(10,2) NOT NULL,
9     statut VARCHAR2(20) CHECK (statut IN ('ACTIF', 'INACTIF', 'CONGE')),
10    email VARCHAR2(100),
11    telephone VARCHAR2(20),
12    poste VARCHAR2(50),
13    id_departement NUMBER,
14    CONSTRAINT fk_emp_dept FOREIGN KEY (id_departement)
15        REFERENCES DEPARTEMENT(id_departement)
16 );

```

Listing 1 – Création de la table EMPLOYE

3.3 Séquences

Des séquences Oracle ont été créées pour générer automatiquement les clés primaires :

```
1 CREATE SEQUENCE seq_employe START WITH 1 INCREMENT BY 1;
2 CREATE SEQUENCE seq_commande START WITH 1 INCREMENT BY 1;
3 CREATE SEQUENCE seq_produit START WITH 1 INCREMENT BY 1;
```

Listing 2 – Création de séquences

4 Programmation PL/SQL

4.1 Procédures Stockées

4.1.1 Calcul des Salaires Mensuels

La procédure `calculer_salaire_mensuel` utilise le SQL dynamique pour calculer automatiquement les salaires de tous les employés actifs en additionnant le salaire de base et toutes les primes.

```
1 CREATE OR REPLACE PROCEDURE calculer_salaire_mensuel(
2     p_mois NUMBER,
3     p_annee NUMBER
4 ) IS
5     v_sql VARCHAR2(4000);
6     v_primes NUMBER;
7     v_salaire_total NUMBER;
8
9     CURSOR c_employes IS
10        SELECT id_employe, salaire_base
11        FROM EMPLOYE
12        WHERE statut = 'ACTIF';
13 BEGIN
14     FOR emp IN c_employes LOOP
15         -- Construction dynamique de la requete
16         v_sql := ...;
17
18         EXECUTE IMMEDIATE v_sql INTO v_primes
19             USING emp.id_employe, p_mois, p_annee;
20
21         v_salaire_total := emp.salaire_base + NVL(v_primes, 0);
22
23         INSERT INTO SALAIRES_MENSUELS
24             VALUES (seq_salaire.NEXTVAL, emp.id_employe, v_salaire_total,
25                 p_mois, p_annee, SYSDATE);
26     END LOOP;
27
28     COMMIT;
29 END;
30 /
```

Listing 3 – Procédure de calcul des salaires

Caractéristiques :

- Utilisation de curseurs explicites
- SQL dynamique avec EXECUTE IMMEDIATE
- Agrégation de données provenant de plusieurs tables
- Gestion des valeurs NULL

4.1.2 Vérification des Stocks

La procédure `verifier_stocks_dynamique` génère des alertes pour les produits dont le stock est inférieur ou égal au seuil d'alerte.

```

1 CREATE OR REPLACE PROCEDURE verifier_stocks_dynamique(
2   p_categorie_produit VARCHAR2 DEFAULT NULL
3 ) IS
4   TYPE t_alerte IS RECORD (
5     produit_nom VARCHAR2(100),
6     entrepot VARCHAR2(50),
7     quantite NUMBER,
8     seuil NUMBER
9   );
10
11   TYPE t_alertes_tab IS TABLE OF t_alerte;
12   v_alertes t_alertes_tab;
13   v_sql VARCHAR2(4000);
14 BEGIN
15   v_sql := ...;
16
17   IF p_categorie_produit IS NOT NULL THEN
18     v_sql := v_sql || 'ANDp.categorie=:cat';
19     EXECUTE IMMEDIATE v_sql BULK COLLECT INTO v_alertes
20       USING p_categorie_produit;
21   ELSE
22     EXECUTE IMMEDIATE v_sql BULK COLLECT INTO v_alertes;
23   END IF;
24
25   FOR i IN 1..v_alertes.COUNT LOOP
26     INSERT INTO ALERTES_STOCK
27       VALUES (seq_alerte.NEXTVAL, v_alertes(i).produit_nom,
28         v_alertes(i).entrepot, v_alertes(i).quantite,
29         v_alertes(i).seuil, SYSDATE);
30   END LOOP;
31
32   COMMIT;
33 END;
34 /

```

Listing 4 – Procédure de vérification des stocks

Caractéristiques :

- Utilisation de types composés (RECORD, TABLE)
- SQL dynamique conditionnel
- BULK COLLECT pour optimiser les performances
- Paramètre optionnel avec valeur par défaut

4.2 Fonctions

4.2.1 Calcul du Total d'une Commande

```

1 CREATE OR REPLACE FUNCTION calculer_total_commande(
2   p_id_commande NUMBER
3 ) RETURN NUMBER IS
4   v_total NUMBER := 0;
5 BEGIN
6   SELECT SUM(quantite * prix_unitaire)
7     INTO v_total

```

```

8      FROM LIGNE_COMMANDE
9      WHERE id_commande = p_id_commande;
10
11     RETURN NVL(v_total, 0);
12 EXCEPTION
13     WHEN NO_DATA_FOUND THEN
14         RETURN 0;
15 END;
16 /

```

Listing 5 – Fonction de calcul du total

4.2.2 Vérification de la Disponibilité

```

1 CREATE OR REPLACE FUNCTION verifier_disponibilite_stock(
2     p_id_produit NUMBER,
3     p_quantite_demandee NUMBER
4 ) RETURN BOOLEAN IS
5     v_quantite_totale NUMBER := 0;
6 BEGIN
7     SELECT SUM(quantite)
8     INTO v_quantite_totale
9     FROM STOCKS
10    WHERE id_produit = p_id_produit;
11
12    RETURN v_quantite_totale >= p_quantite_demandee;
13 EXCEPTION
14     WHEN NO_DATA_FOUND THEN
15         RETURN FALSE;
16 END;
17 /

```

Listing 6 – Fonction de vérification du stock

4.3 Package de Gestion des Commandes

Un package PL/SQL a été développé pour encapsuler la logique métier liée aux commandes.

4.3.1 Spécification du Package

```

1 CREATE OR REPLACE PACKAGE pkg_gestion_commandes IS
2     PROCEDURE creer_commande(
3         p_id_client NUMBER,
4         p_liste_produits VARCHAR2,
5         p_id_commande OUT NUMBER
6     );
7
8     FUNCTION calculer_total_commande(
9         p_id_commande NUMBER
10    ) RETURN NUMBER;
11
12    PROCEDURE valider_stock_disponible(
13        p_id_commande NUMBER,
14        p_disponible OUT BOOLEAN
15    );
16
17    PROCEDURE generer_facture(

```

```

18         p_id_commande NUMBER
19     );
20 END pkg_gestion_commandes;
21 /

```

Listing 7 – Spécification du package

Avantages du Package :

- Encapsulation de la logique métier
- Réutilisabilité du code
- Maintenance facilitée
- Performance améliorée (compilation unique)

4.4 Triggers

4.4.1 Trigger d'Audit

Le trigger `audit_modifications_sensibles` enregistre automatiquement toutes les modifications sur la table des salaires.

```

1 CREATE OR REPLACE TRIGGER audit_modifications_sensibles
2 AFTER INSERT OR UPDATE OR DELETE ON SALAIRES_MENSUELS
3 FOR EACH ROW
4 DECLARE
5     v_operation VARCHAR2(10);
6     v_table_name VARCHAR2(50) := 'SALAIRES_MENSUELS';
7     v_user VARCHAR2(50) := USER;
8 BEGIN
9     IF INSERTING THEN
10         v_operation := 'INSERT';
11         INSERT INTO AUDIT_LOG
12             VALUES (seq_audit.NEXTVAL, v_table_name, v_operation,
13                     :NEW.id_employe, NULL, :NEW.montant_total, v_user, SYSDATE);
14
15     ELSIF UPDATING THEN
16         v_operation := 'UPDATE';
17         INSERT INTO AUDIT_LOG
18             VALUES (seq_audit.NEXTVAL, v_table_name, v_operation,
19                     :NEW.id_employe, :OLD.montant_total, :NEW.montant_total,
20                     v_user, SYSDATE);
21
22     ELSIF DELETING THEN
23         v_operation := 'DELETE';
24         INSERT INTO AUDIT_LOG
25             VALUES (seq_audit.NEXTVAL, v_table_name, v_operation,
26                     :OLD.id_employe, :OLD.montant_total, NULL, v_user, SYSDATE);
27     END IF;
28 END;
29 /

```

Listing 8 – Trigger d'audit

4.4.2 Trigger de Mise à Jour Automatique

```

1 CREATE OR REPLACE TRIGGER maj_montant_commande
2 AFTER INSERT OR UPDATE OR DELETE ON LIGNE_COMMANDE
3 FOR EACH ROW
4 DECLARE

```

```

5      v_id_commande NUMBER;
6      v_total NUMBER;
7  BEGIN
8      IF DELETING THEN
9          v_id_commande := :OLD.id_commande;
10     ELSE
11         v_id_commande := :NEW.id_commande;
12     END IF;
13
14     SELECT SUM(quantite * prix_unitaire)
15     INTO v_total
16     FROM LIGNE_COMMANDE
17     WHERE id_commande = v_id_commande;
18
19     UPDATE COMMANDE
20     SET montant_total = NVL(v_total, 0)
21     WHERE id_commande = v_id_commande;
22 END;
23 /

```

Listing 9 – Trigger de mise à jour du montant

4.4.3 Trigger de Validation

```

1  CREATE OR REPLACE TRIGGER verif_stock_avant_validation
2  BEFORE UPDATE OF statut ON COMMANDE
3  FOR EACH ROW
4  WHEN (NEW.statut = 'VALIDEE')
5  DECLARE
6      v_stock_insuffisant EXCEPTION;
7      v_count NUMBER;
8  BEGIN
9      SELECT COUNT(*)
10     INTO v_count
11     FROM LIGNE_COMMANDE lc
12     WHERE lc.id_commande = :NEW.id_commande
13     AND NOT EXISTS (
14         SELECT 1
15         FROM STOCKS s
16         WHERE s.id_produit = lc.id_produit
17         HAVING SUM(s.quantite) >= lc.quantite
18         GROUP BY s.id_produit
19     );
20
21     IF v_count > 0 THEN
22         RAISE v_stock_insuffisant;
23     END IF;
24 EXCEPTION
25     WHEN v_stock_insuffisant THEN
26         RAISE_APPLICATION_ERROR(-20001,
27             'Stock insuffisant pour valider cette commande');
28 END;
29 /

```

Listing 10 – Trigger de vérification du stock

5 Gestion des Utilisateurs et des Privilèges

5.1 Création des Utilisateurs

Quatre types d'utilisateurs ont été créés avec des privilèges différents :

1. **admin_db** : Administrateur de la base (DBA)
2. **app_user** : Utilisateur de l'application (accès complet)
3. **lecteur_rapports** : Utilisateur en lecture seule
4. **gestionnaire_cmd** : Gestionnaire des commandes

```

1 CREATE USER admin_db IDENTIFIED BY Admin123
2 DEFAULT TABLESPACE USERS
3 TEMPORARY TABLESPACE TEMP
4 QUOTA UNLIMITED ON USERS;
5
6 CREATE USER app_user IDENTIFIED BY App123
7 DEFAULT TABLESPACE USERS
8 TEMPORARY TABLESPACE TEMP
9 QUOTA 100M ON USERS;
10
11 CREATE USER lecteur_rapports IDENTIFIED BY Lecteur123
12 DEFAULT TABLESPACE USERS
13 TEMPORARY TABLESPACE TEMP
14 QUOTA 10M ON USERS;
```

Listing 11 – Création d'utilisateurs

5.2 Création des Rôles

Des rôles ont été créés pour faciliter la gestion des privilèges :

- **role_admin** : Gestion complète de la base
- **role_ventes** : Gestion des ventes et commandes
- **role_consultation** : Consultation en lecture seule
- **role_logistique** : Gestion des stocks et entrepôts

```

1 CREATE ROLE role_admin;
2 CREATE ROLE role_ventes;
3 CREATE ROLE role_consultation;
4 CREATE ROLE role_logistique;
```

Listing 12 – Création de rôles

5.3 Attribution des Privilèges

5.3.1 Privilèges pour le Rôle Administrateur

```

1 GRANT CREATE SESSION TO role_admin;
2 GRANT CREATE TABLE TO role_admin;
3 GRANT CREATE VIEW TO role_admin;
4 GRANT CREATE PROCEDURE TO role_admin;
5 GRANT CREATE TRIGGER TO role_admin;
6
7 GRANT SELECT, INSERT, UPDATE, DELETE ON DEPARTEMENT TO role_admin;
8 GRANT SELECT, INSERT, UPDATE, DELETE ON EMPLOYE TO role_admin;
9 GRANT SELECT, INSERT, UPDATE, DELETE ON CLIENT TO role_admin;
```

Listing 13 – Privilèges administrateur

5.3.2 Privilèges pour le Rôle Ventes

```
1 GRANT CREATE SESSION TO role_ventes;
2 GRANT SELECT, INSERT, UPDATE ON CLIENT TO role_ventes;
3 GRANT SELECT, INSERT, UPDATE ON COMMANDE TO role_ventes;
4 GRANT SELECT, INSERT, UPDATE ON LIGNE_COMMANDE TO role_ventes;
5 GRANT SELECT ON PRODUIT TO role_ventes;
6 GRANT EXECUTE ON pkg_gestion_commandes TO role_ventes;
```

Listing 14 – Privilèges ventes

5.4 Attribution des Rôles aux Utilisateurs

```
1 GRANT role_admin TO admin_db;
2 GRANT DBA TO admin_db;
3 GRANT role_admin TO app_user;
4 GRANT role_consultation TO lecteur_rapports;
5 GRANT role_ventes TO gestionnaire_cmd;
```

Listing 15 – Attribution des rôles

5.5 Vues

Des vues ont été créées pour masquer les informations sensibles :

```
1 CREATE OR REPLACE VIEW V_EMPLOYE_PUBLIC AS
2 SELECT id_employe, matricule, nom, prenom,
3        poste, id_departement, statut
4 FROM EMPLOYE;
5
6 GRANT SELECT ON V_EMPLOYE_PUBLIC TO role_consultation;
```

Listing 16 – Vue pour masquer les données sensibles

6 Optimisation des Performances

6.1 Stratégie d'Optimisation

La stratégie d'optimisation mise en place comprend :

1. Création d'index appropriés
2. Utilisation de vues
3. Optimisation des requêtes SQL
4. Surveillance des performances

6.2 Indexation

6.2.1 Index sur les Clés Étrangères

```
1 CREATE INDEX idx_emp_dept ON EMPLOYE(id_departement);
2 CREATE INDEX idx_prod_fourn ON PRODUIT(id_fournisseur);
3 CREATE INDEX idx_stock_prod ON STOCKS(id_produit);
4 CREATE INDEX idx_cmd_client ON COMMANDE(id_client);
5 CREATE INDEX idx_ligne_cmd ON LIGNE_COMMANDE(id_commande);
```

Listing 17 – Index sur les clés étrangères

Impact : Amélioration des performances des jointures

6.2.2 Index Composites

```
1 CREATE INDEX idx_cmd_date_statut ON COMMANDE(date_commande, statut);
2 CREATE INDEX idx_emp_statut_dept ON EMPLOYE(statut, id_departement);
```

Listing 18 – Index composites

Impact : Optimisation des requêtes avec conditions multiples

6.2.3 Index Basés sur Fonction

```
1 CREATE INDEX idx_client_nom_upper ON CLIENT(UPPER(nom));
```

Listing 19 – Index fonctionnels

Impact : Recherches insensibles à la casse plus rapides

7 Tests

7.1

7.1.1 Création de Commande

```
1 DECLARE
2     v_id_cmd NUMBER;
3     v_disponible BOOLEAN;
4 BEGIN
5     pkg_gestion_commandes.creer_commande(
6         p_id_client => 1,
7         p_liste_produits => '1:2,2:5',
8         p_id_commande => v_id_cmd
9     );
10
11     DBMS_OUTPUT.PUT_LINE('Commande_cree:' || v_id_cmd);
12
13     pkg_gestion_commandes.valider_stock_disponible(v_id_cmd, v_disponible);
14
15     IF v_disponible THEN
16         DBMS_OUTPUT.PUT_LINE('Stock_disponible');
17     ELSE
18         DBMS_OUTPUT.PUT_LINE('Stock_insuffisant');
19     END IF;
20 END;
21 /
```

Listing 20 – Test de commande

7.1.2 Calcul de Salaire

```
1 BEGIN
2     calculer_salaire_mensuel(12, 2024);
3 END;
4 /
5
6 SELECT e.nom, e.prenom, s.montant_total
7 FROM SALAIRES_MENSUELS s
8 JOIN EMPLOYE e ON s.id_employe = e.id_employe
9 WHERE s.mois = 12 AND s.annee = 2024;
```

Listing 21 – Test de calcul salaire

7.1.3 Vérification des Stocks

```
1 BEGIN
2     verifier_stocks_dynamique('Accessoires');
3 END;
4 /
5
6 SELECT * FROM ALERTES_STOCK
7 WHERE date_alerte = TRUNC(SYSDATE);
```

Listing 22 – Test de vérification stocks

7.2

7.2.1 Test des Privilèges

```
1 -- En tant que lecteur_rapports
2 CONNECT lecteur_rapports/Lecteur123
3
4 -- reussir
5 SELECT * FROM V_EMPLOYE_PUBLIC;
6
7 -- echouer
8 UPDATE EMPLOYE SET salaire_base = 6000 WHERE id_employe = 1;
```

Listing 23 – Vérification des privilèges

8 Conclusion

8.1 Objectifs Atteints

Ce projet d'administration de bases de données a permis de mettre en pratique l'ensemble des concepts étudiés dans le module.

1. **Conception** : Un modèle de données normalisé et cohérent a été élaboré
2. **Implémentation** : Une base de données Oracle et fonctionnelle
3. **Programmation PL/SQL** : Des procédures, fonctions, packages et triggers
4. **Sécurité** : Un système de gestion des utilisateurs et privilèges est en place
5. **Sauvegarde** : Une stratégie de sauvegarde et restauration
6. **Optimisation**

8.2 Compétences Acquisées

À travers ce projet, nous avons développé les compétences suivantes :

- l'architecture Oracle
- Conception de bases de données relationnelles
- Programmation en PL/SQL
- Gestion des curseurs
- Création de triggers

- Administration des utilisateurs et des privilèges
- Mise en place de stratégies de sauvegarde
- Optimisation des performances
- Utilisation de Docker pour l'environnement Oracle

8.3 Points Forts du Projet

- **Architecture solide** : Modèle de données normalisé
- **Automatisation** : Calculs automatisés (salaires, alertes)
- **Sécurité** : Gestion fine des privilèges
- **Performances** : Optimisations
- **Maintenabilité** : Code structuré et documenté
- **Résilience** : Stratégie de sauvegarde

8.4 Difficultés Rencontrées et Solutions

1. **Gestion du SQL dynamique**
 - *Problème* : Complexité de construction des requêtes dynamiques
2. **Optimisation des jointures**
 - *Solution* : Création d'index
3. **Configuration Docker**

8.5 Améliorations Possibles

1. **Ajout de fonctionnalités**
 - Gestion des retours de commandes
 - Prévisions de stocks avec IA
 - Tableau de bord temps réel
2. **Sécurité**
 - Chiffrement des données sensibles
 - Masquage de données pour les environnements
 - Authentification

8.6 Perspectives

Ce projet constitue une base solide pour un système de Gestion des Ressources d'Entreprise. Les prochaines étapes pourraient inclure :

- Migration vers une architecture microservices
- Mise en place d'une stratégie Big Data

8.7 Conclusion

Ce projet d'administration de bases de données nous a permis d'appliquer concrètement les concepts théoriques du cours. La conception, l'implémentation et l'administration d'un système nous ont donné une vision globale des responsabilités d'un administrateur de bases de données.

Bibliographie

Références

SAHMI, I. (2025). *Support de cours - Administration des Bases de Données*. ENSA Berrechid, Université Hassan 1er.

Oracle Corporation. *Oracle Database 21c Documentation*. <https://docs.oracle.com/en/database/oracle/oracle-database/21/>